Glossary:

MVP: minimal viable product
ARTA: shorthand for the risk mitigation strategies
PSI: potentially shippable increment
Feature: same as backlog item
Product backlog item: a planned feature
Sprint backlog item: a feature being worked on in a past or current sprint
DoD: definition of done: a measure of implementation quality has all the features previously agreed upon been implemented + tested + are they working without errors.
Skateboard: Proof of concept version demonstrating viability of a complete project.
Wheel: Proof of concept of a (small) part of the project, parts of which are thus unimplemented. This must be individually testable and runnable.
CR: code review
RA: risk assessment
RMP: Risk mitigation plan

Capstone Assignment 1

Submit this form as a MS Word docx file to Amathuba.
Also check it in to your Gitlab repo as a MS word docx file, so that team members, tutors, and clients have access to it.

Initial project plan [10% of total mark] *Do not edit this document after submission to Amathuba*			
Project name:	MatchPoint		
Team nickname	TLDR		
Team names (first/last)	Thomas	Luke	Danny
	Kabalin	Bryant	Gebhard
Student numbers	KBLTHO001	BRYLUK001	GBHDAN001
Role	Backend, Data, and DevOps	Frontend: Capture & Scoring	Frontend: Analytics & Accounts
Project description *1 or 2 lines*	MatchPoint is a tennis statistics tracking that lets coaches capture live match data and turn it into player performance analytics.		
Planned programming language(s)	Languages: TypeScript, Python, SQL Platform: Progressive Web App (React)		
Git url	https://gitlab.cs.uct.ac.za/tldr/matchpoint		
Sprint length	2 weeks		
Describe non-backlog/non-coding work *(filling in forms is not 'work')*			
Work item 1	**Research the game of tennis and its associated match rules**		
Person responsible	Luke Bryant		

Work item 2	**Research tennis player statistics in order to determine which would be of interest to a coach and their player**		
Person responsible	Danny Gebhard		
Work item 3	**Research the technology stack required for the project**		
Person responsible	Thomas Kabalin		
Work item 4	**Research visualisation techniques and heuristics, and what generally appeals to people in the way of UI (learning from apps that are loved for their UI)**		
Person responsible	Danny Gebhard		
Work Item 5	**Research and wireframe the match capture workflow**		
Person responsible	Luke Bryant		
Work Item 6	**Generate and/or acquire seed data**		
Person responsible	Thomas Kabalin		
Risk analysis			
Describe Risk 1	**Team member is unavailable due to illness, etc.**		
Likelihood	Unlikely	Impact	Expensive (High)
ARTA strategy	Transfer		
Mitigations	Cross train team members and maintain documentation so others can pick up work. Build buffer time into project schedule.		
Describe Risk 2	**SQL injection allowing unauthorized data access or manipulation**		
Likelihood	Likely	Impact	Expensive (High)
ARTA strategy	Avoid		
Mitigations	Use parameterized queries/ORM (SQLAlchemy) exclusively - no raw string-concatenated queries; input validation and sanitization at the API layer		
Describe Risk 3	**Weak/broken authentication allowing account takeover**		
Likelihood	Likely	Impact	Expensive (High)
ARTA strategy	Avoid		
Mitigations	Avoid custom-built cryptography or session logic entirely by using established, audited libraries/frameworks for password hashing		

	(bcrypt/argon2), session management, and OTP-based MFA; enforce a strong password policy; rate-limit login attempts		
Describe Risk 4	**Does not comply with POPI Act and similar rule / laws**		
Likelihood	Unlikely	Impact	Expensive (High)
ARTA strategy	Transfer		
Mitigations	Host on RackZar, a POPIA-compliant hosting provider, to transfer infrastructure-level data protection obligations; conduct an internal compliance review early in the project; implement data minimization and consent mechanisms; define a clear privacy policy and terms of service		
Describe Risk 5	**Loss/corruption of source code**		
Likelihood	Unlikely	Impact	Expensive (High)
ARTA strategy	Transfer		
Mitigations	Use version control (git) as well as a remote repository backup (GitLab). Furthermore, ensure regular commits and access control to the repository.		
Describe Risk 6	**Minor UI / cosmetic bug reported by users during user-testing**		
Likelihood	Likely	Impact	Cheap (Low)
ARTA strategy	Reduce		
Mitigations	Use cross-browser and cross-device testing during developement. Set up continuous deployment to reduce the time taken to fix any bugs.		
Describe Risk 7	**Broken access control / IDOR (Insecure Direct Object Reference) - a coach able to access another coach's player or match data by manipulating IDs in requests**		
Likelihood	Likely	Impact	Expensive (High)
ARTA strategy	Avoid		
Mitigations	Server-side authorisation checks on every endpoint, all queries scoped to the authenticated coach's own records, automated tests verifying cross-coach access is rejected (401/403)		
Describe Risk 8	**Unauthorized database access due to misconfiguration (e.g. open DB ports, default credentials)**		
Likelihood	Likely	Impact	Expensive (High)
ARTA strategy	Avoid		

Mitigations	Database access restricted to the application layer only (no public-facing DB access), strong unique credentials, firewall/network-level access rules, encryption at rest		
Describe Risk 9	**Data integrity failure - corrupted or inconsistent records (e.g. from concurrent writes, sync errors in offline capture)**		
Likelihood	Likely	Impact	Expensive (High)
ARTA strategy	Avoid		
Mitigations	Avoid concurrent-write conflicts by design - restrict each match to a single active writer/session at a time, and disallow offline edits to existing records (offline mode only queues new point creation never edits); use PostgreSQL's ACID transactions, row-level locking, and constraints (NOT NULL, UNIQUE, CHECK, foreign keys) to make invalid or conflicting writes structurally impossible at the database level, not just handled after the fact		

Project backlog items	
Add as many backlog items as needed	
Backlog item 1	
Name	**Data model**
Description	Shippable increment: migrated PostgreSQL schema plus SQLAlchemy models and test factories
Priority	High
Main developer	Thomas Kabalin
Estimated time	0.5 weeks
Acceptance criteria (DoD):	
Functionality	Schema and models must accurately represent coach, player, match, and point data, structured for use by the API layer. Must support input validation and access control at the database level to guard against SQL injection attacks and unauthorized data access
Test cases	• Given a migration is run, expect it to complete cleanly with no errors or data loss • Given the test factories are run, expect them to generate valid records for all entities (coach, player, match, point) • Given invalid or malformed input is submitted, expect it to be rejected at the model/DB layer with no injection-style input reaching raw queries • Given a request attempts direct/unauthenticated access to the database, expect it to be denied, with access restricted to the application layer only
Client remarks	
Approved by client	*signature*

Backlog item 2	
Name	**Player, match and point APIs**
Description	As a coach, I want to manage players, create matches with their metadata, and have captured points accepted reliably.
Priority	High
Main developer	Thomas Kabalin
Estimated time	0.7 weeks
Acceptance criteria (DoD):	
Functionality	Must reliably create/read/update players and matches and accept point data with server-side validation. Every endpoint must require authentication and only expose a coach's own data.
Test cases	• Given valid player/match data is submitted, expect CRUD operations to succeed • Given an invalid point submission is made, expect the API to return a clear validation error • Given an unauthenticated request or a request for another coach's data, expect the API to reject it with a 401/403 response
Client remarks	
Approved by client	*signature*
Backlog Item 3	
Name	**Accounts And Access**
Description	As a coach, I want my own account and sign-in so that my players and match data are private to me and only reachable by me.
Priority	Middle
Main developer	Danny Gebhard
Estimated time	0.7 weeks
Acceptance criteria (DoD)	
Functionality	Must let a coach register, log in, and log out, with passwords hashed, sessions correctly isolating each coach's data, and multi-factor authentication supported to reduce the risk of broken authentication
Test cases	• Given valid registration/login details are submitted, expect the account to be created/logged in successfully; given invalid credentials, expect them to be rejected • Given a password is stored, expect it to be hashed and never stored in plaintext • Given Coach A attempts to access Coach B's data, expect the request to be denied • Given a coach logs in with correct credentials, expect a multi-factor authentication request to trigger correctly
Client remarks	

Approved by client	*signature*
Backlog Item 4	
Name	**Scoring Engine**
Description	As a coach, I want the app to work out the score from the points I record so that I never track games and sets by hand.
Priority	High
Main developer	Luke Bryant
Estimated time	0.5 weeks
Acceptance criteria (DoD)	
Functionality	Must correctly calculate game/set/match scores from recorded points, matching standard scoring rules exactly (verified against known scenarios like deuce/tiebreak).
Test cases	• Given a standard sequence of points is recorded, expect the game/set/match score to be calculated correctly • Given an edge case scenario (deuce, tiebreak, advantage) occurs, expect the score to resolve correctly per standard rules
Client remarks	
Approved by client	*signature*
Backlog Item 5	
Name	**Match Data Capture**
Description	As a coach at the court, I want to set up a match in a few taps and then record every point in no more than three taps so that the whole courtside workflow keeps up with live play.
Priority	Middle
Main developer	Luke Bryant
Estimated time	0.9 weeks
Acceptance criteria (DoD)	
Functionality	Must let a coach set up a match easily and record each point in no more than 3 taps accurately and in real time. Access must be restricted to the authenticated coach's own match
Test cases	• Given a coach sets up a new match, expect setup to be completed in ≤3 taps • Given a coach records a point, expect it to be completed in ≤3 taps • Given points are entered rapidly in succession, expect each point to be correctly attributed with no duplicates • The UI should not allow the coach to enter a state that is impossible as per the rules of tennis (e.g. asking for a forced /

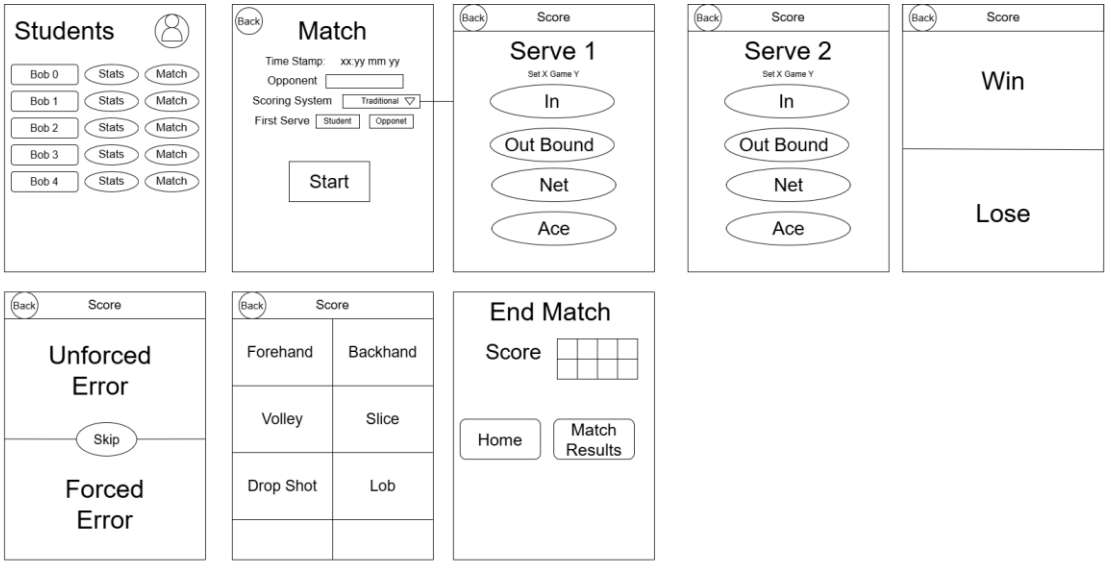
	unforced error classification when the player has not yet served)
Client remarks	
Approved by client	*Signature*
Backlog Item 6	
Name	**Statistics And Analytics**
Description	As a coach, I want the statistics computed from captured points, shown as a post-match summary and as a dashboard of trends across matches and opponents.
Priority	Low
Main developer	Danny Gebhard
Estimated time	0.7 weeks
Acceptance criteria (DoD)	
Functionality	Must accurately compute post-match summaries and cross-match trend dashboards from captured data, visible only to the coach who owns that data.
Test cases	• Given match data has been captured, expect the post-match summary to match manually calculated values • Given data exists across multiple matches/opponents, expect the dashboard to correctly display and aggregate trends • Given a coach views their statistics, expect them to see only their own data and not another coach's • For each statistic, there should be unit tests to ensure that it is computed correctly, even with edge-case data
Client remarks	
Approved by client	*signature*
Backlog Item 7	
Name	**Offline Capture**
Description	As a coach on a court with no signal, I want capture to keep working and sync when connectivity returns so that I never lose a match's data.
Priority	Low
Main developer	Thomas Kabalin
Estimated time	0.7 weeks
Acceptance criteria (DoD)	
Functionality	Must keep recording points with no connection, then sync accurately with no data loss once connectivity returns, without exposing cached data to other apps/users.
Test cases	• Given the device has no network connection, expect the app to continue recording points successfully

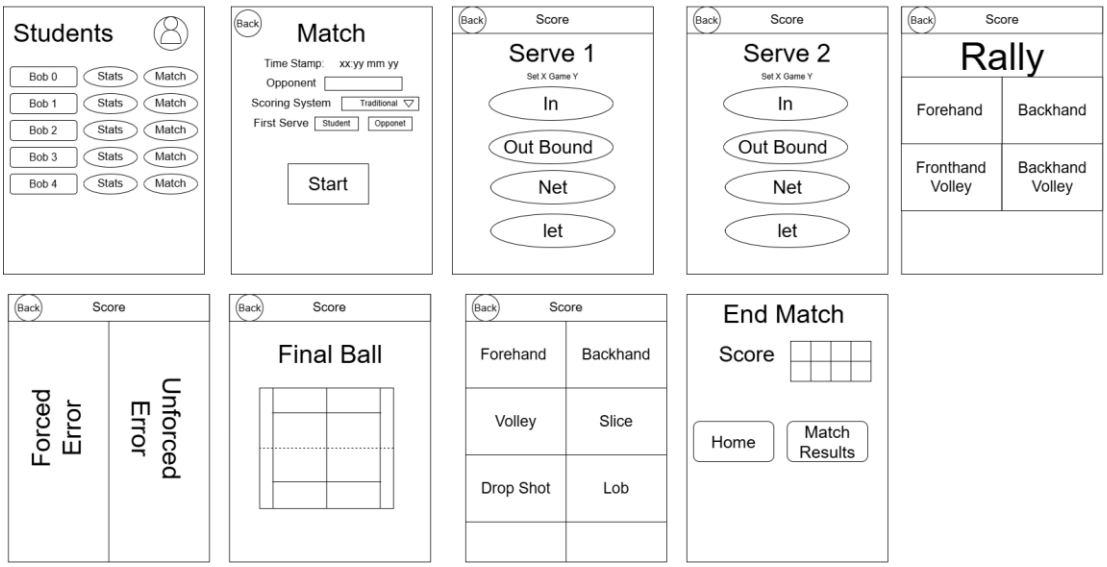
	• Given connectivity returns after offline recording, expect queued data to sync correctly with no loss or duplication
Client remarks	
Approved by client	*signature*
Backlog Item 8	
Name	**Home Page & App Navigation Flow**
Description	As a coach, I want a home page that gives me a clear overview when I log in, with easy navigation to my players, matches, and stats, so that I can quickly get to what I need without hunting through menus.
Priority	Middle
Main Developer	Danny Gebhard
Estimated Time	0.5 weeks
Acceptance criteria (DoD)	
Functionality	Must present an authenticated coach with a home page summarizing key info (e.g., recent matches, quick actions) and provide clear, consistent navigation to all core app pages (players, matches, stats, account), restricted to that coach's own data only.
Test cases	• Given an authenticated coach logs in, expect the home page to load and display only that coach's own data • Given a coach with recent matches recorded, expect the home page to display a summary of those recent matches • Given a coach taps/clicks a navigation link (e.g., Players, Matches, Stats), expect the app to route to the correct corresponding page • Given an unauthenticated user attempt to access the home page directly (e.g., via URL), expect the app to redirect them to the login page • Given a coach navigates between core pages (Players, Matches, Stats, Settings), expect all links to work with no dead ends or broken routes • Given a coach on a typical mobile connection loads the home page, expect it to fully load within an acceptable time (e.g., ≤2 seconds)
Client remarks	
Approved by client	*signature*
Backlog Item 9	
Name	**Test data generation**

Description	As a developer, I want realistic seed/test data for coaches, players, matches, and points so that the team can properly test functionality, edge cases, and performance without relying on manual data entry.
Priority	Middle
Main developer	Luke Bryant
Estimated time	0.4 weeks
Acceptance criteria (DoD)	
Functionality	Must generate realistic, structured test data covering coaches, players, matches, and points (including edge cases like deuce/tiebreak scenarios and double faults), reusable across environments without polluting production data.
Test cases	• Given the test data script is run, expect it to populate the database with valid coach, player, match, and point records with no errors • Given generated match data, expect it to include edge case scenarios (e.g. deuce, tiebreak, double faults) so scoring logic can be fully tested • Given generated match data, expect it to include edge case scenarios (e.g. deuce, tiebreak, double faults) so scoring logic can be fully tested • Given the test data script is run in a non-production environment, expect it to have no effect on production data • Given the test data script is run multiple times, expect it to not create duplicate or conflicting records
Client remarks	
Approved by client	*signature*
GenAI declaration	
Did you use GenAI	***Y****/N*
Describe use	Claude Opus 5 was used in developing initial test case ideas for all Backlog items. These were then edited and reworded by the team. Claude Opus 4.6 was used to write the tech stack documentation, but all decisions were made by the team. Claude Sonnet 5 was used to assist separating our 3 main backlog items into smaller, more manageable partitions. Claude Sonnet 5 to provide relative time estimates of the various backlog items, to gain insight into the complexity of each task. We then took those relative estimates and assigned them absolute values.

	Claude Sonnet 5 used to help reword and improve sentence structure across the document. Claude Design was used for the sign in, home, and analytics dashboards wireframes.

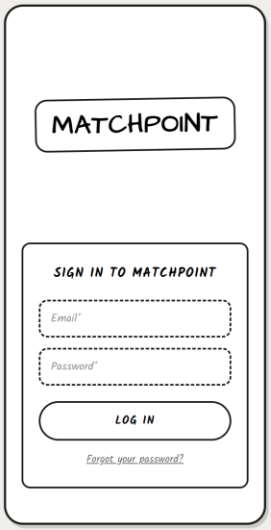
Wireframe for match capture flow



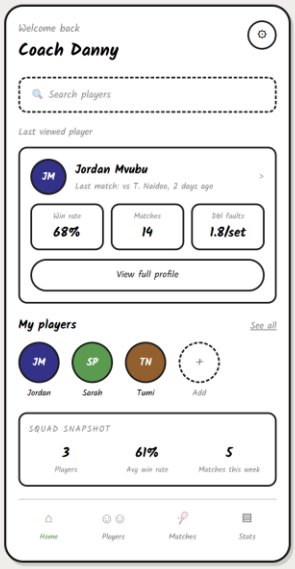


Wireframe for sign in, home, and analytics dashboards

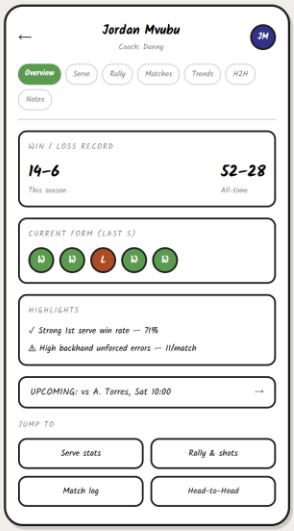
1a Sign In



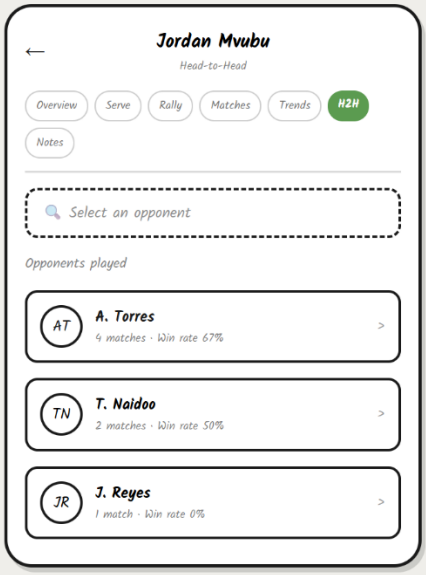
1b Coach Home — my players



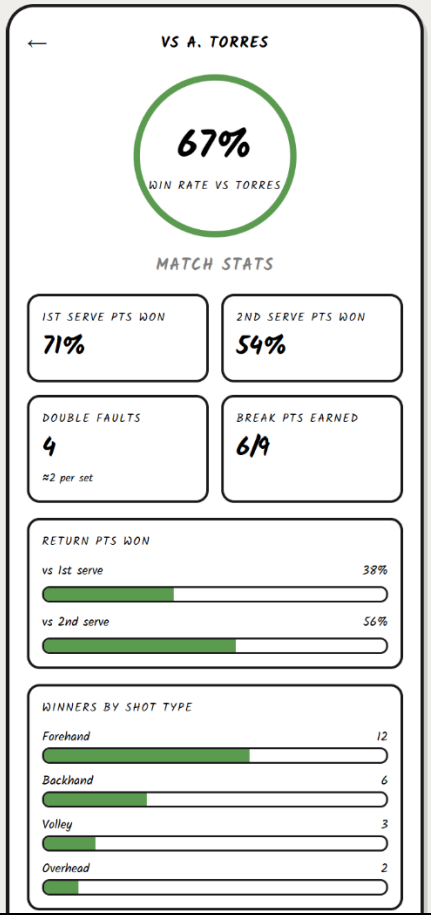
1d Player Profile — Overview tab (from 1b)



1e Player Profile — Head-to-Head tab (from 1d)



1c H2H detail — Jordan vs Torres (from 1e)



Appendix: Tech Stack

Selection criteria

Choices were evaluated against four criteria:

1. **Boring and well-trodden -** large tutorial/StackOverflow/LLM training corpus. AI agents are dramatically more productive on mainstream stacks, and well-established tools have fewer undocumented edge cases to stumble into.
2. **Type-safety end-to-end -** compile-time errors catch mistakes early, and give AI agents a tight feedback loop to iterate against.
3. **Team skills -** Python-leaning backend experience; everyone touching React for the first time.
4. **Time-to-value -** the stack should let us spend hours on the interesting problems (capture UX, scoring engine, analytics), not on framework wrestling.

Where a choice of version exists, we default to the most widely adopted release rather than the latest, since community support and documentation coverage peak there.

Frontend

Category	Choice
Language	TypeScript 5.9
UI framework	React 19
Bundler	Vite 8 (PWA via vite-plugin-pwa)
CSS framework	Tailwind CSS 4
Component library	shadcn/ui (Base UI)
Routing	TanStack Router
API codegen	@hey-api/openapi-ts
Server state	TanStack Query
Offline outbox	*TBD* -TanStack Query mutations or Dexie
Package manager	pnpm 11.18 workspace
Lint / format	ESLint 10 + Prettier 3
Type checking	tsc
Tests	Vitest 4 + Testing Library 16 + Playwright

Rationale

React + Vite SPA. A Single Page Application (SPA) loads once and handles all navigation in the browser without full page reloads from the server. This matters here because a Progressive Web App (PWA) - a web app that can be installed to the home screen and work offline - boots from a cached app shell, which is inherently SPA behaviour. Server-Side Rendering (SSR), where the server generates each page's HTML on request, provides nothing for an app that must function without a server connection. vite-plugin-pwa is the most mature PWA tooling in the React ecosystem. No Node server to run - the built app is static files behind a reverse proxy, pairing cleanly with a Python API on a cheap VPS. Alternatives considered: Next.js (brings App Router/RSC concepts and a Node runtime we don't need;

actively fights the offline app-shell model unless used as a static export, at which point it's Vite with extra steps).

PWA over native. No app-store friction, which is critical for rapid usability-testing iterations with the client. iOS is now genuinely workable: installed PWAs are exempt from Safari's 7-day storage eviction, storage quota is up to ~60% of disk, and Screen Wake Lock works. Two managed caveats: no Background Sync API (sync must flush while foregrounded) and no screen-orientation lock (capture UI works in both orientations). Alternatives considered: React Native/Expo (second codebase, TestFlight distribution slows testing cycles, team has no RN experience); Flutter/Dart (same app-store distribution friction, slowing usability-testing iterations with the client).

Tailwind CSS. Required by shadcn/ui, which uses Tailwind utility classes for all component styling. Beyond that dependency, Tailwind's utility-first approach works well with AI agents and avoids the naming/scoping overhead of CSS Modules.

shadcn/ui on Base UI. Copy-in components give full control over styling - essential for the highly custom capture screen -while the dashboard gets prebuilt components (tables, cards, dialogs) for free. Base UI is now shadcn's default primitives layer, backed by the Radix/Floating-UI creators. Agents handle shadcn well. Alternatives considered: shadcn on Radix (no longer the default for new projects); hand-rolled Tailwind only (would mean rebuilding dialogs/menus/accessibility by hand).

TanStack Router. The strongest type safety of the options we looked at including typed search params (useful for dashboard filter state in shareable URLs), natural pairing with TanStack Query. React Router v8 was the conservative alternative -either works; TanStack won on the type-safety.

TanStack Query. Built-in networkMode/onlineManager for offline handling, cache persistence to IndexedDB via persistQueryClient, optimistic updates. Used for server-state reads and optimistic UI.

@hey-api/openapi-ts. FastAPI's auto-generated OpenAPI 3.1 spec feeds directly into typed TypeScript client generation with a TanStack Query plugin. This is now FastAPI's own recommended path.

Backend

Category	Choice
Language	Python 3.12
API framework	FastAPI 0.141 + Pydantic 2
ORM	SQLAlchemy 2 (sync)
Migrations	Alembic
Package manager	uv
Lint / format	Ruff 0.16
Type checking	Pyright
Auth	*TBD*
Tests	pytest 9

Rationale

Python + FastAPI over an all-TypeScript backend. The offline-first requirement forces scoring logic into the client, so the backend's jobs shrink to: auth, durable storage, sync ingestion, and analytics queries. Python is the team's language strength and the natural home for the analytics side (aggregation queries, potential pandas for trend computation). FastAPI is the most mainstream Python API framework with first-class auto-OpenAPI generation, feeding the typed TS client pipeline. Large LLM training corpus. The strongest alternative was an all-TypeScript backend: one language everywhere, scoring engine shared via a workspace package, single toolchain. It lost on team skills (everyone learning the backend stack from scratch during the project) and on losing Python's analytics ergonomics.

SQLAlchemy 2 (sync) + Alembic. The standard pair; SQLAlchemy 2.0's unified API is well-known to LLMs. Sync is deliberate: FastAPI runs sync routes in a threadpool, and our load is club-scale - async adds complexity for zero benefit here while keeping a later async migration mechanical. Alternative considered: SQLModel (still 0.0.x after five years, no API-stability guarantee; our analytics queries would outgrow its sweet spot).

uv + Ruff + Pyright. The modern Python toolchain: uv for package/project management, Ruff for lint and format (replaces multiple tools), Pyright for type checking. All are fast, well-maintained, and LLM-friendly.

Infrastructure

Category	Choice
Database	PostgreSQL 18
Local DB hosting	Docker Compose
CI	GitLab CI

Rationale

PostgreSQL over SQLite and MySQL. PostgreSQL handles concurrent access cleanly (SQLite is single-writer) and is the standard choice for production web applications. SQLite was the lightweight alternative but its single-writer ceiling made it less appropriate for a real application. MySQL was the other mainstream option; PostgreSQL was chosen over it based on existing team experience. All schema lives in Alembic migrations from day one, keeping switching costs low regardless.

Docker Compose for local dev. Gives dev/prod parity with minimal overhead. The same Compose file describes the production deployment on the VPS.

GitLab CI. Dictated by the university's GitLab instance (gitlab.cs.uct.ac.za) being the required code host for submission. The CI runner is self-hosted on an off-campus server since the university instance does not provide shared runners.